
Documentacao de Projeto

para o sistema PataAmiga

Versao 1.1

Projeto de sistema elaborado pelo aluno Renato Douglas como parte da disciplina Projeto de Software.

23 de maio de 2026

Historico de Revisoes

Nome	Data	Razoes para Mudanca	Versao
Renato Douglas	23/05/2026	Versao inicial - entrega do Trabalho 2 da disciplina Projeto de Software	1.1

Tabela de Conteudo

Historico de Revisoes	2
1. Introducao.....	5
1.1. Sobre o PataAmiga.....	5
1.2. Stack tecnologica adotada	5
2. Modelos de Usuario e Requisitos.....	5
2.1. Descricao de Atores	5
2.2. Modelo de Casos de Uso	6
2.2.1. Relacionamentos entre Casos de Uso.....	7
2.3. Diagrama de Sequencia do Sistema	9
3. Modelos de Projeto.....	11
3.1. Arquitetura	11
3.1.1. Organizacao de pacotes do backend.....	11
3.1.2. C4 Nivel 1 - Contexto	11
3.1.3. C4 Nivel 2 - Containers.....	12
3.2. Diagrama de Componentes e Implantacao	13
3.2.1. Diagrama de Componentes (C4 Nivel 3)	13
3.2.2. Diagrama de Implantacao.....	14
3.3. Diagrama de Classes	16
3.3.1. Principais classes do dominio	16
3.4. Diagramas de Sequencia	17
3.4.1. Realizacao por caso de uso.....	17
3.5. Diagramas de Comunicacao	28
3.6. Diagramas de Estados	33
3.6.1. Resumo dos estados	33
3.6.2. Principais transicoes.....	33
4. Modelos de Dados	34
4.1. Diagrama Entidade-Relacionamento.....	35
4.1.1. Visao geral das tabelas	35
4.2. Estrategia de Mapeamento Objeto-Relacional	36
4.2.1. Convencoes gerais	36
4.2.2. Mapeamento da Abordagem C (Usuario + Perfis).....	36
4.2.3. Mapeamento dos principais relacionamentos	37
4.2.4. Estrategias de busca (FetchType) e cascata	38

4.2.5. Versionamento de schema.....	38
-------------------------------------	----

1. Introdução

Este documento agrega: (1) a elaboração e revisão de modelos de domínio e (2) modelos de projeto para o sistema PataAmiga. A referência principal para a descrição geral do problema, domínio e requisitos do sistema é o documento de especificação que descreve a visão de domínio do sistema.

1.1. Sobre o PataAmiga

O PataAmiga é uma plataforma web desenvolvida para centralizar e otimizar a gestão de ONGs dedicadas ao resgate e adoção de animais. O sistema acompanha o ciclo de vida completo de cada animal - desde o resgate inicial, passando pelos cuidados veterinários, até a adoção responsável - integrando todos os atores envolvidos (adotantes, voluntários, veterinários, coordenadores e doadores) em uma única solução.

ONGs do setor frequentemente operam com processos manuais e descentralizados (planilhas, grupos de mensagens, cadernos físicos), o que dificulta o controle, compromete a rastreabilidade e limita a capacidade de atendimento. O PataAmiga resolve esse problema oferecendo uma plataforma centralizada que:

- Registra e acompanha cada animal desde o resgate até a adoção;
- Organiza o histórico veterinário completo de cada animal;
- Gerencia o processo de adoção com etapas formalizadas;
- Conecta adotantes a animais disponíveis com busca por perfil;
- Permite que doadores acompanhem o impacto de suas contribuições;
- Fornece ao coordenador da ONG visibilidade operacional e relatórios gerenciais.

1.2. Stack tecnológica adotada

Camada	Tecnologia
Frontend	Angular 17 + TypeScript
Backend	Java 21 + Spring Boot 3.x
Banco de Dados	PostgreSQL 16
ORM	Hibernate / JPA
Mapeamento DTO <-> Entity	MapStruct
Autenticação	JWT + Spring Security
Containerização	Docker + Docker Compose

2. Modelos de Usuário e Requisitos

2.1. Descrição de Atores

O PataAmiga possui cinco atores que interagem com o sistema, agrupados em externos (clientes finais) e internos (operadores da ONG). A tabela a seguir resume papel, tipo e responsabilidades principais de cada um.

Ator	Tipo	Responsabilidades
Adotante	Externo / Primario	Buscar animais disponiveis, submeter solicitacao de adocao, acompanhar o processo e registrar pos-adocao
Voluntario	Interno / Operacional	Registrar animais resgatados, agendar atendimentos veterinarios, atualizar status e apoiar eventos
Veterinario	Interno / Especialista	Emitir laudos clinicos, registrar prontuarios, aplicar vacinas e procedimentos, liberar animais para adocao
Coordenador da ONG	Interno / Administrativo	Gerenciar equipe, aprovar ou rejeitar adocoos, controlar recursos, emitir relatorios gerenciais
Doador	Externo / Suporte	Realizar doacoes pontuais ou recorrentes, acompanhar uso dos recursos e visualizar relatorios de impacto

Decisao de modelagem (Abordagem C). Embora os atores sejam apresentados de forma separada na visao UML, no modelo de dados existe uma unica entidade Usuario central com cinco perfis 1:1 opcionais (PerfilAdotante, PerfilDoador, PerfilVoluntario, PerfilVeterinario, PerfilCoordenador). Uma mesma pessoa pode acumular qualquer combinacao de papeis. O detalhamento aparece na Secao 3.3 (Diagrama de Classes) e na Secao 4 (Modelos de Dados).

2.2. Modelo de Casos de Uso

Os 14 casos de uso do sistema estao agrupados em 5 pacotes funcionais. Cada caso de uso recebe um ID UC-NN usado como referencia cruzada nas demais secoes.

ID	Caso de Uso	Ator(es) Primario(s)	Pacote
UC-01	Solicitar Adocao	Adotante	Adocao
UC-02	Registrar Animal Resgatado	Voluntario	Gestao de Animais
UC-03	Registrar Atendimento Veterinario	Veterinario	Atendimento Veterinario
UC-04	Buscar Animal Disponivel	Adotante	Adocao
UC-05	Acompanhar Processo de Adocao	Adotante	Adocao
UC-06	Registrar Pos-Adocao	Adotante	Adocao
UC-07	Atualizar Status do Animal	Voluntario	Gestao de Animais

UC-08	Agendar Atendimento Veterinario	Voluntario	Gestao de Animais
UC-09	Emitir Laudo de Liberacao para Adocao	Veterinario	Atendimento Veterinario
UC-10	Aprovar/Rejeitar Adocao	Coordenador da ONG	Adocao
UC-11	Gerenciar Voluntarios	Coordenador da ONG	Administracao
UC-12	Emitir Relatorios Gerenciais	Coordenador da ONG	Administracao
UC-13	Realizar Doacao	Doador	Doacoes
UC-14	Acompanhar Destinacao das Doacoes	Doador	Doacoes

2.2.1. Relacionamentos entre Casos de Uso

Relacionamento	Descricao
UC-03 «include» UC-07	Todo atendimento veterinario registra uma transicao de status (ex.: RESGATADO -> EM_TRATAMENTO).
UC-09 «include» UC-07	A emissao do laudo de liberacao move o animal para DISPONIVEL.
UC-10 «include» UC-07	A aprovacao move para EM_PROCESSO_ADOCAO/ADOTADO; a rejeicao mantem em DISPONIVEL.

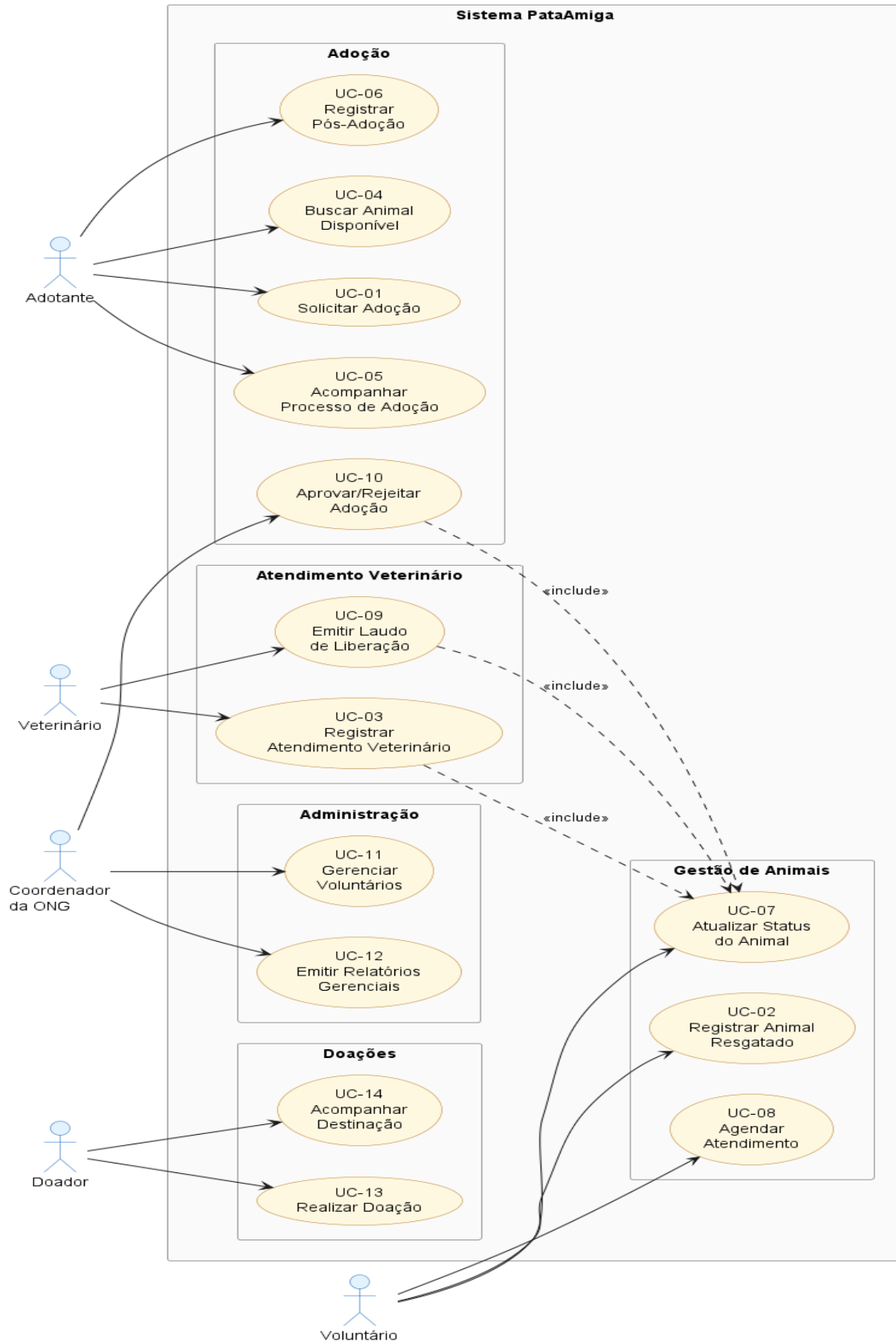


Figura 1 - Diagrama de Casos de Uso do PataAmiga

2.3. Diagrama de Sequencia do Sistema

Apresenta-se a seguir o Diagrama de Sequencia do Sistema (SSD) do PataAmiga, em visao caixa-preta e ponta a ponta. Os cinco atores interagem com :Sistema por meio das operacoes de sistema, cobrindo o ciclo de vida do animal (resgate -> tratamento -> liberacao -> adocao -> pos-adocao) e o fluxo paralelo de doacoes. Por ser caixa-preta, o diagrama omite as camadas internas; a realizacao interna (caixa-branca) de cada operacao e detalhada nos diagramas de sequencia da Secao 3.4 (Figuras 8 a 21).

Operacao de Sistema	Ator	Caso de Uso
registrarAnimalResgatado(dados)	Voluntario	UC-02
registrarAtendimento(idAnimal, dados)	Veterinario	UC-03 «include» UC-07
emitirLaudoLiberacao(idAnimal, parecer)	Veterinario	UC-09 «include» UC-07
buscarAnimaisDisponiveis(especie, porte, sexo)	Adotante	UC-04
submeterSolicitacaoAdocao(idAnimal, formularioInteresse)	Adotante	UC-01
aprovarAdocao(idAdocao) / concluirAdocao(idAdocao)	Coordenador	UC-10 «include» UC-07
registrarPosAdocao(idAdocao, relato)	Adotante	UC-06
realizarDoacao(valor, tipo, metodo)	Doador	UC-13

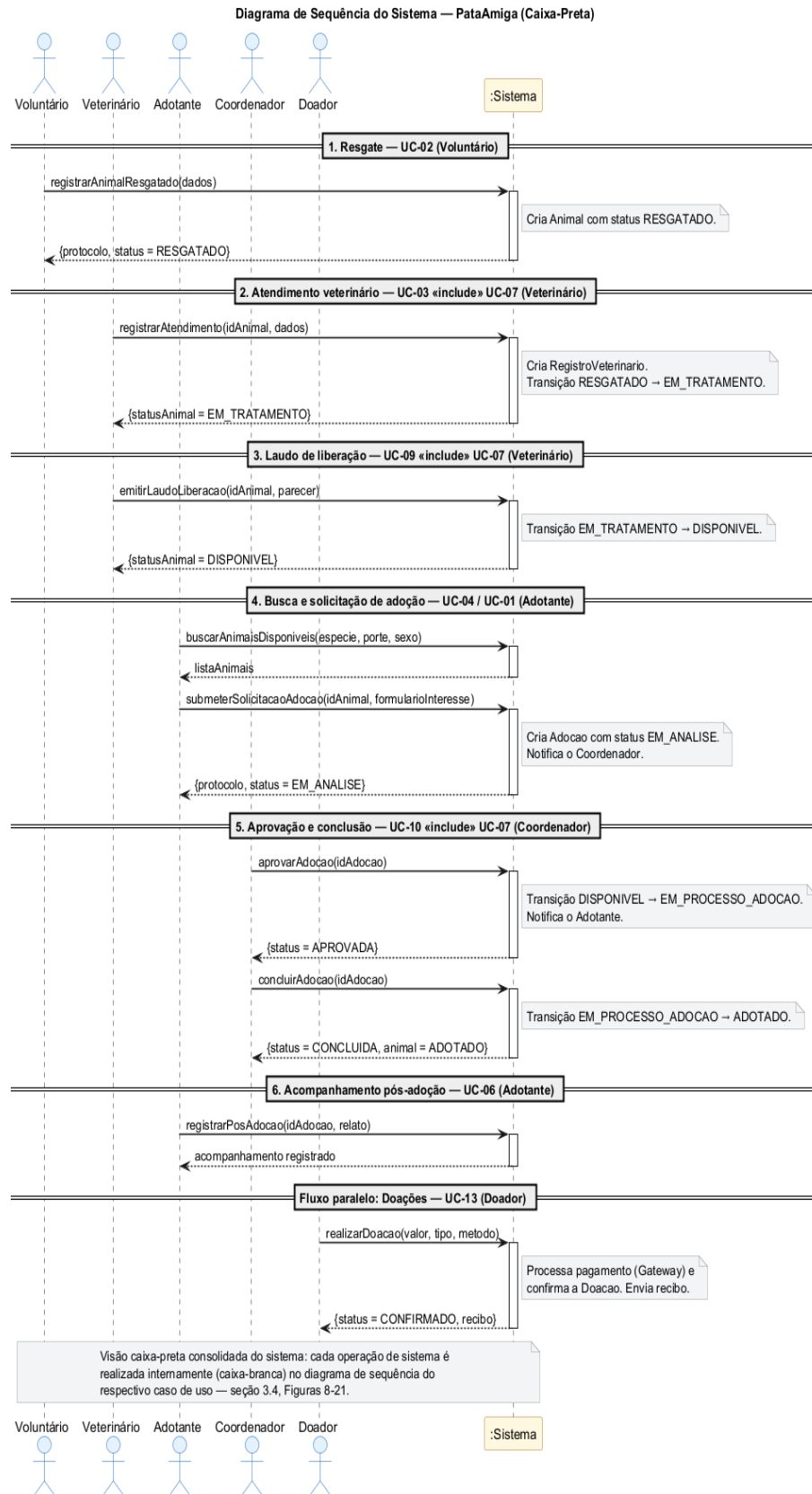


Figura 2 - Diagrama de Sequencia do Sistema do PataAmiga (caixa-preta, ponta-a-ponta)

3. Modelos de Projeto

3.1. Arquitetura

O PataAmiga adota uma arquitetura monolitica modular em camadas, adequada para o porte de uma ONG de pequeno e medio porte. A arquitetura e descrita usando o C4 Model em tres niveis (Contexto, Containers e Componentes).

3.1.1. Organizacao de pacotes do backend

O codigo do backend segue a organizacao tipica de uma aplicacao Spring Boot baseada em camadas, sob a raiz br.pucminas.pataamiga:

controller/	@RestController, mapeamento de rotas REST
service/	@Service, regras de negocio
repository/	interfaces Spring Data JPA (@Repository)
model/	entidades JPA (@Entity, @Table)
dto/	
request/	payloads de entrada (AnimalRequestDTO...)
response/	payloads de saida (AnimalResponseDTO...)
mapper/	conversao DTO <-> Entity via MapStruct (@Mapper)
exception/	exceptions customizadas + @RestControllerAdvice
config/	SecurityConfig, OpenApiConfig, CorsConfig
security/	JwtFilter, JwtUtil, UserDetailsServiceImpl

Padroes adotados: Repository Pattern, Service Layer, DTO (request/response), Mapper (MapStruct), Global Exception Handling, Spring Security com RBAC por perfil de ator.

3.1.2. C4 Nivel 1 - Contexto

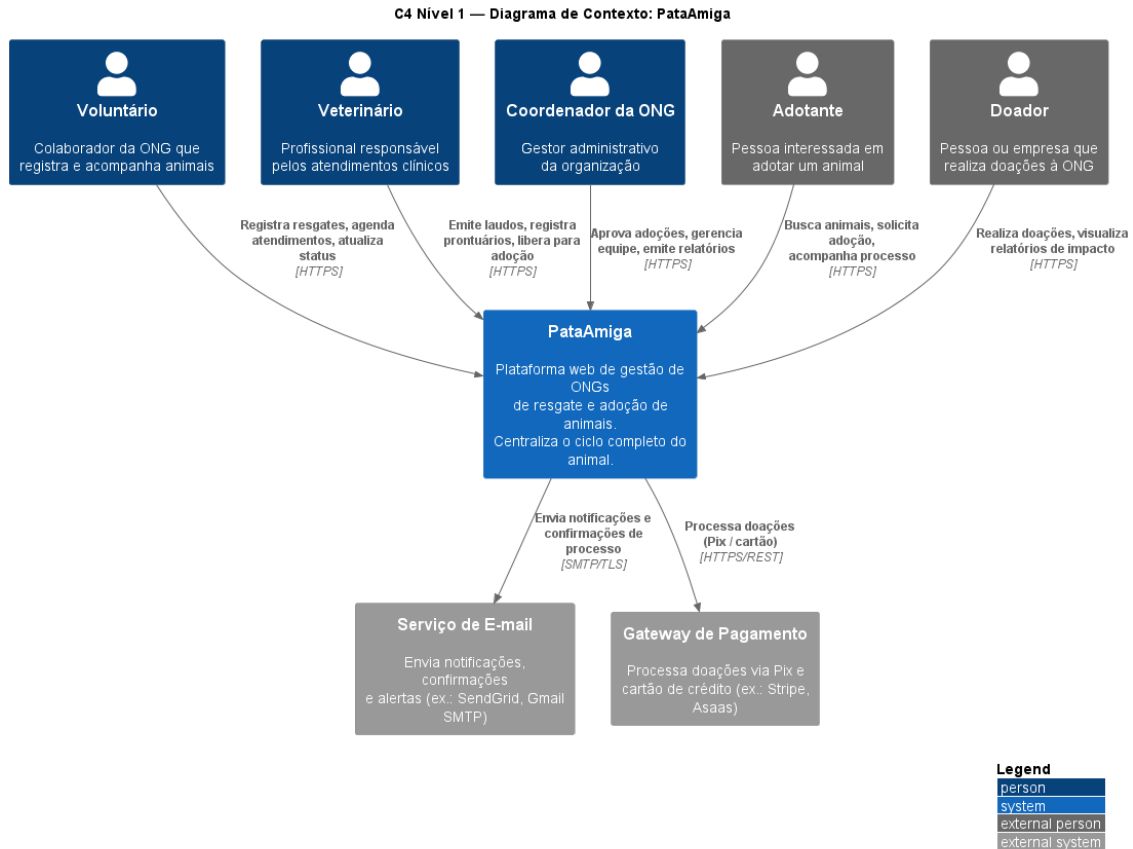


Figura 3 - C4 Nível 1: Diagrama de Contexto do PataAmiga

3.1.3. C4 Nível 2 - Containers

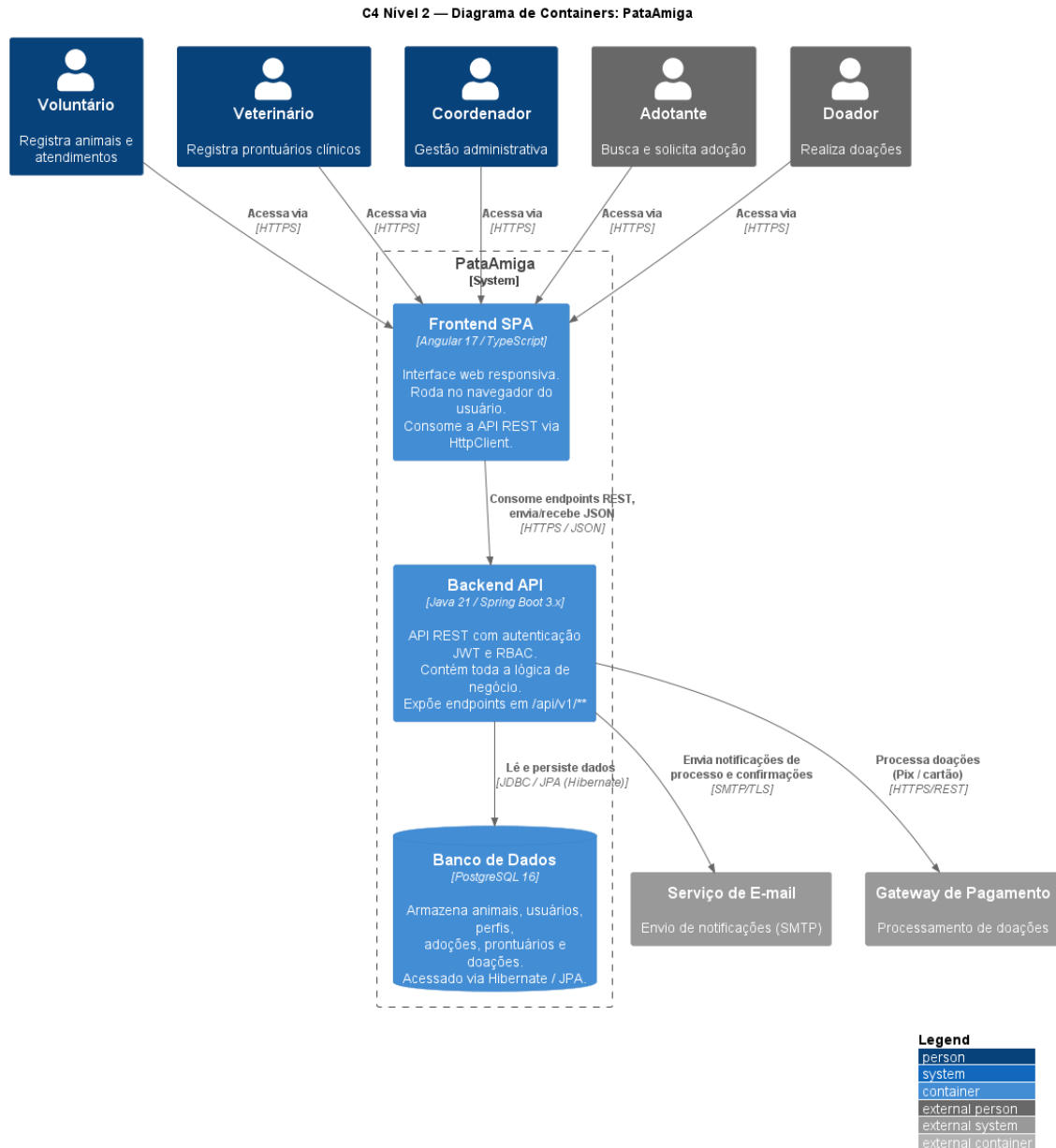


Figura 4 - C4 Nível 2: Diagrama de Containers do PataAmiga

3.2. Diagrama de Componentes e Implantacao

O diagrama de componentes corresponde ao C4 Nível 3, detalhando os componentes internos do backend Spring Boot. O diagrama de implantacao mostra onde cada container é alocado na infraestrutura Docker.

3.2.1. Diagrama de Componentes (C4 Nível 3)

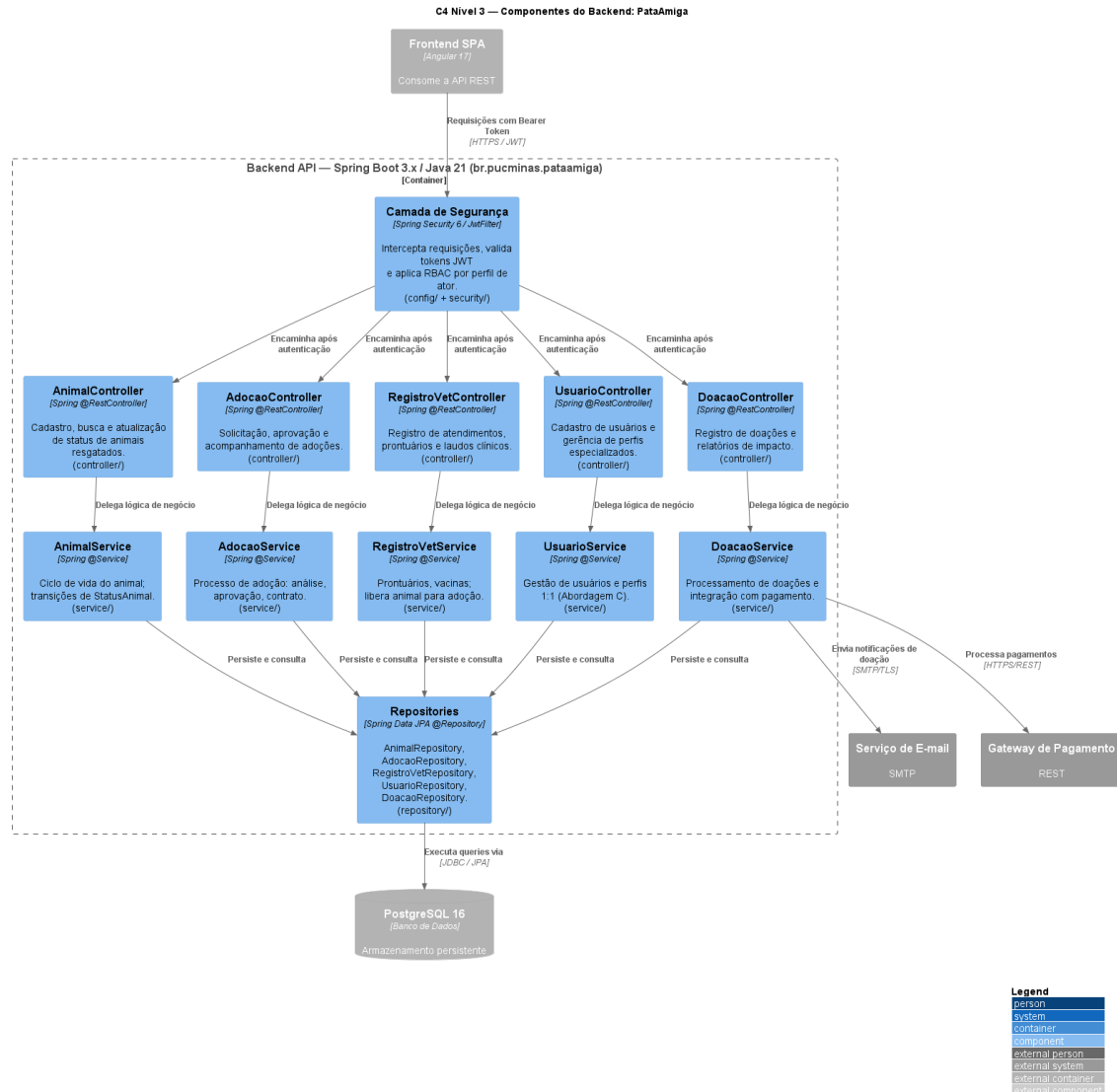


Figura 5 - C4 Nível 3: Componentes do Backend PataAmiga

3.2.2. Diagrama de Implantacao

A implantacao e orquestrada por Docker Compose (arquivo docker-compose.yml na raiz do projeto). Tres containers compartilham a rede bridge pataamiga-net em um host Linux com Docker Engine, e dois sistemas externos sao consumidos pela API.

Container	Imagem Docker	Portas (host:container)	Responsabilidade	Volume
pataamiga-frontend	nginx:alpine	4200:80	Servir o build estatico do SPA Angular 17	-
pataamiga-backend	eclipse-temurin:21-jre	8080:8080	API REST Spring Boot 3.x	-

			(/api/v1/**) com JWT e RBAC	
pataamiga-db	postgres:16-alpine	5432:5432	Banco PostgreSQL com schema pataamiga	pataamiga-pgdata

Sistema externo	Protocolo	Uso
Servico de E-mail (SMTP)	SMTP/TLS	Notificacoes de adocao e confirmacoes de doacao
Gateway de Pagamento	HTTPS/REST	Processamento de doacoes via Pix e cartao

Ordem de inicializacao: pataamiga-db (com healthcheck) -> pataamiga-backend (depends_on: db) -> pataamiga-frontend (depends_on: backend).

Diagrama de Implantação — PataAmiga (Docker Compose)

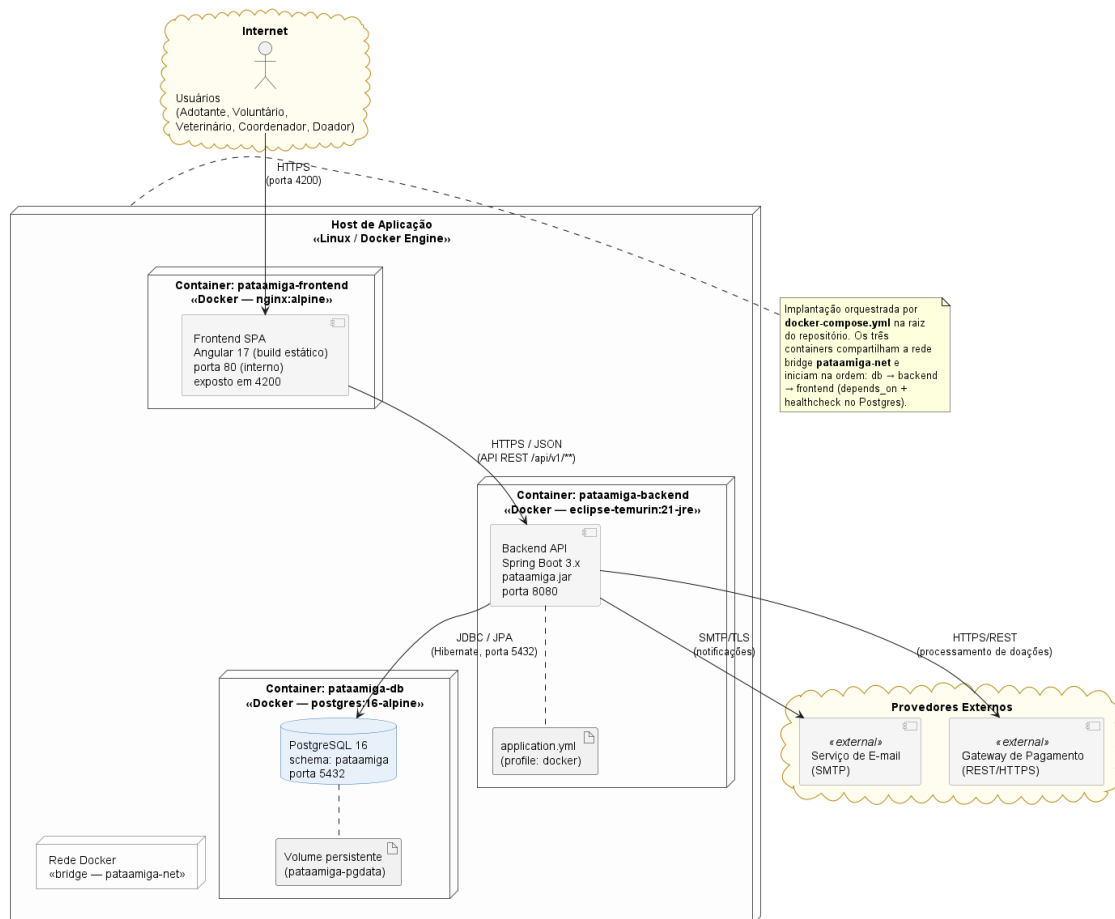


Figura 6 - Diagrama de Implantacao do PataAmiga (Docker Compose)

3.3. Diagrama de Classes

O modelo de dominio adota a Abordagem C para usuarios: uma entidade Usuario central concentra os dados pessoais (nome, e-mail, CPF, telefone, endereco, senha), e cinco perfis especializados - PerfilAdotante, PerfilDoador, PerfilVoluntario, PerfilVeterinario e PerfilCoordenador - estendem Usuario por composicao 1:1 opcional. Uma mesma pessoa pode acumular qualquer combinacao de papeis sem duplicar dados de cadastro.

3.3.1. Principais classes do dominio

Classe	Tipo	Papel no modelo
Usuario	Entidade central	Identidade comum a todos os atores; responsavel por autenticacao
Perfil (abstrata)	Superclasse	Define ativo, dataAtivacao e operacoes genericas dos perfis
PerfilAdotante / Doador / Voluntario / Veterinario / Coordenador	Perfis 1:1	Especializam Usuario com atributos e operacoes especificas de cada papel
Animal	Entidade central do negocio	Estado controlado pelo enum StatusAnimal; relaciona-se com prontuarios e adocoes
RegistroVeterinario	Entidade	Cada atendimento veterinario registrado; liberaParaAdocao aciona transicao de status
Adocao	Entidade	Processo de adocao com ciclo de vida EM_ANALISE -> ... -> CONCLUIDA/REJEITADA
Doacao	Entidade	Aporte pontual ou recorrente associado a um PerfilDoador
Relatorio	Entidade	Saida agregada gerada por um PerfilCoordenador
StatusAnimal, StatusAdocao, TipoDoacao, TipoRelatorio, TipoAtendimento, Especie, Sexo, Porte	Enums	Estados e classificadores do dominio

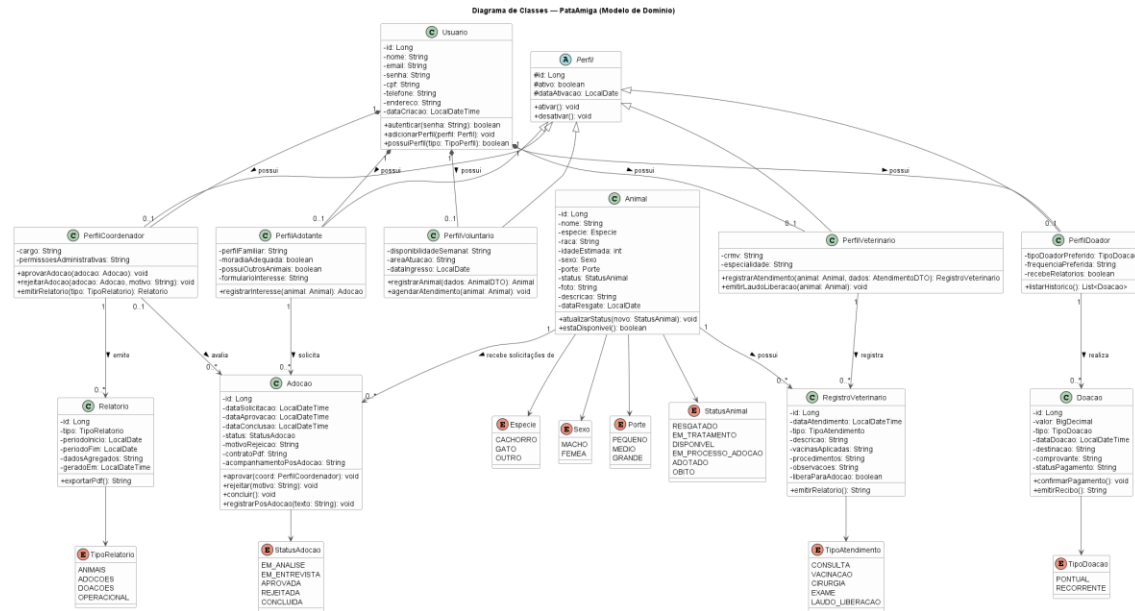


Figura 7 - Diagrama de Classes do Modelo de Domínio do PataAmiga

3.4. Diagramas de Sequencia

Esta seção apresenta a realização interna (caixa-branca) das operações de sistema definidas no Diagrama de Sequência do Sistema da Seção 2.3. Diferente do SSD - que trata o sistema como uma caixa-preta (ator <-> :Sistema) -, cada diagrama de sequência abre o :Sistema e detalha a colaboração entre as camadas da arquitetura - SPA Angular -> JwtFilter -> Controller -> Service -> Repository -> PostgreSQL -, reutilizando os mesmos componentes do C4 Nível 3 (Figura 5). Apresenta-se a realização individual de cada um dos 14 casos de uso (UC-01 a UC-14).

3.4.1. Realização por caso de uso

A tabela a seguir mapeia cada caso de uso aos componentes que participam da sua realização (Controllers, Services e Repositories), seguida pelos respectivos diagramas de sequência caixa-branca.

Fig.	Caso de Uso	Controllers	Services	Repositories
8	UC-01 Solicitar Adocao	AnimalController, AdocaoController	AnimalService, AdocaoService	AnimalRepository, AdocaoRepository
9	UC-02 Registrar Animal Resgatado	AnimalController	AnimalService	AnimalRepository, RegistroVetRepository
10	UC-03 Registrar Atendimento Veterinario	AnimalController, RegistroVetController	AnimalService, RegistroVetService	AnimalRepository, RegistroVetRepository
11	UC-04 Buscar Animal Disponível	AnimalController	AnimalService	AnimalRepository

12	UC-05 Acompanhar Processo de Adocao	AdocaoController	AdocaoService	AdocaoRepository
13	UC-06 Registrar Pos-Adocao	AdocaoController	AdocaoService, AnimalService	AdocaoRepository, AnimalRepository
14	UC-07 Atualizar Status do Animal	AnimalController	AnimalService	AnimalRepository
15	UC-08 Agendar Atendimento Veterinario	RegistroVetController	RegistroVetService	AnimalRepository, RegistroVetRepository
16	UC-09 Emitir Laudo de Liberacao	RegistroVetController	RegistroVetService, AnimalService	AnimalRepository, RegistroVetRepository
17	UC-10 Aprovar/Rejeitar Adocao	AdocaoController	AdocaoService, AnimalService	AdocaoRepository, AnimalRepository
18	UC-11 Gerenciar Voluntarios	UsuarioController	UsuarioService	UsuarioRepository
19	UC-12 Emitir Relatorios Gerenciais	DoacaoController	DoacaoService	AdocaoRepository, DoacaoRepository, RelatorioRepository
20	UC-13 Realizar Doacao	DoacaoController	DoacaoService	DoacaoRepository
21	UC-14 Acompanhar Destinacao das Doacoes	DoacaoController	DoacaoService	DoacaoRepository

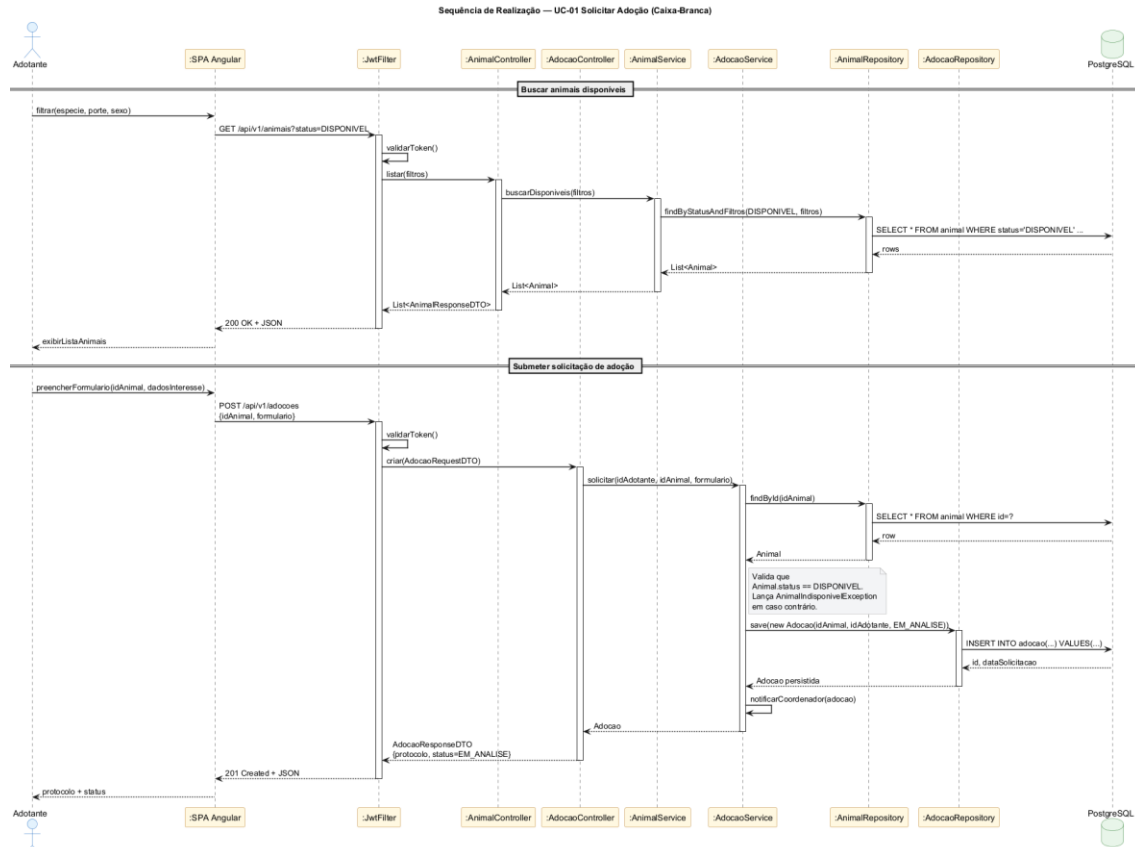


Figura 8 - Realizacao UC-01 Solicitar Adocao

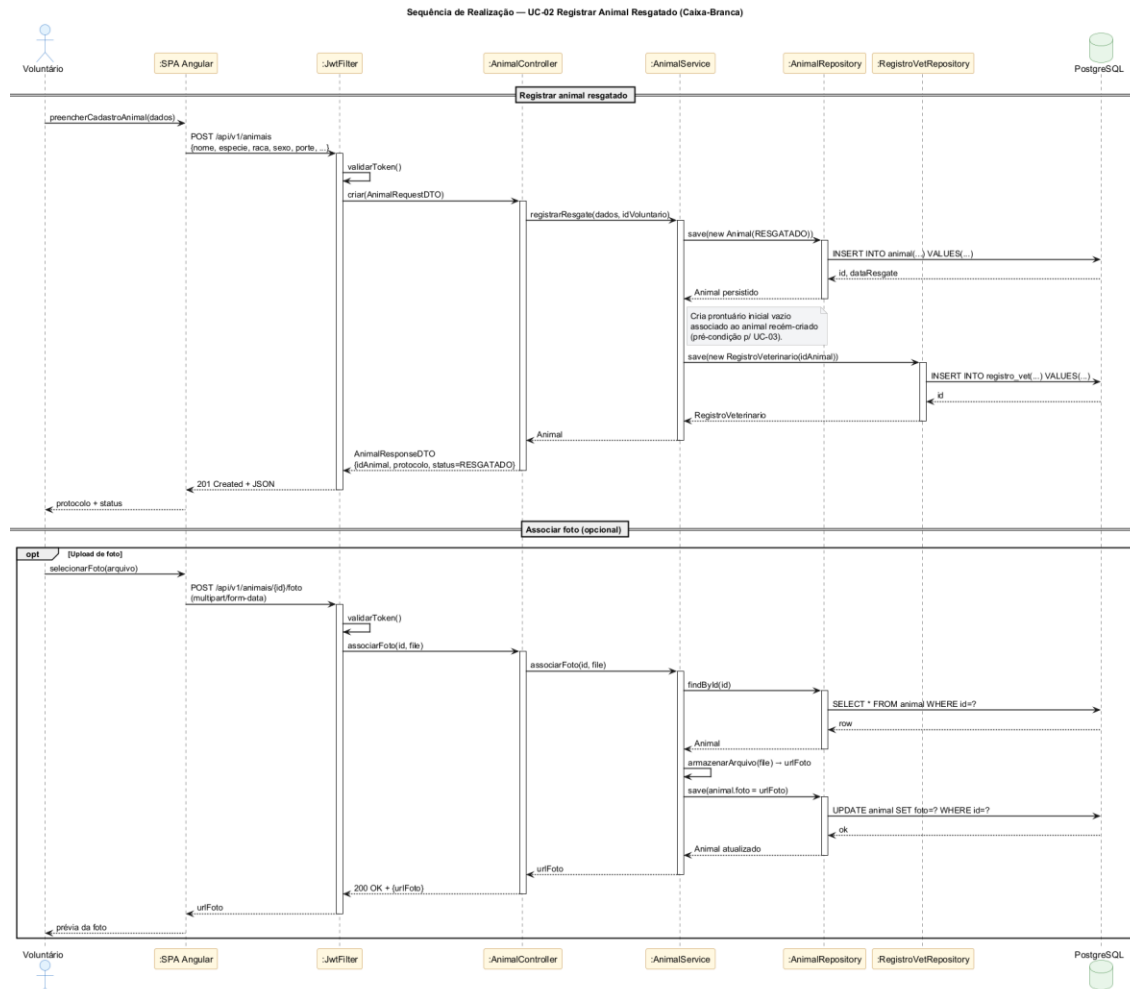


Figura 9 - Realização UC-02 Registrar Animal Resgatado

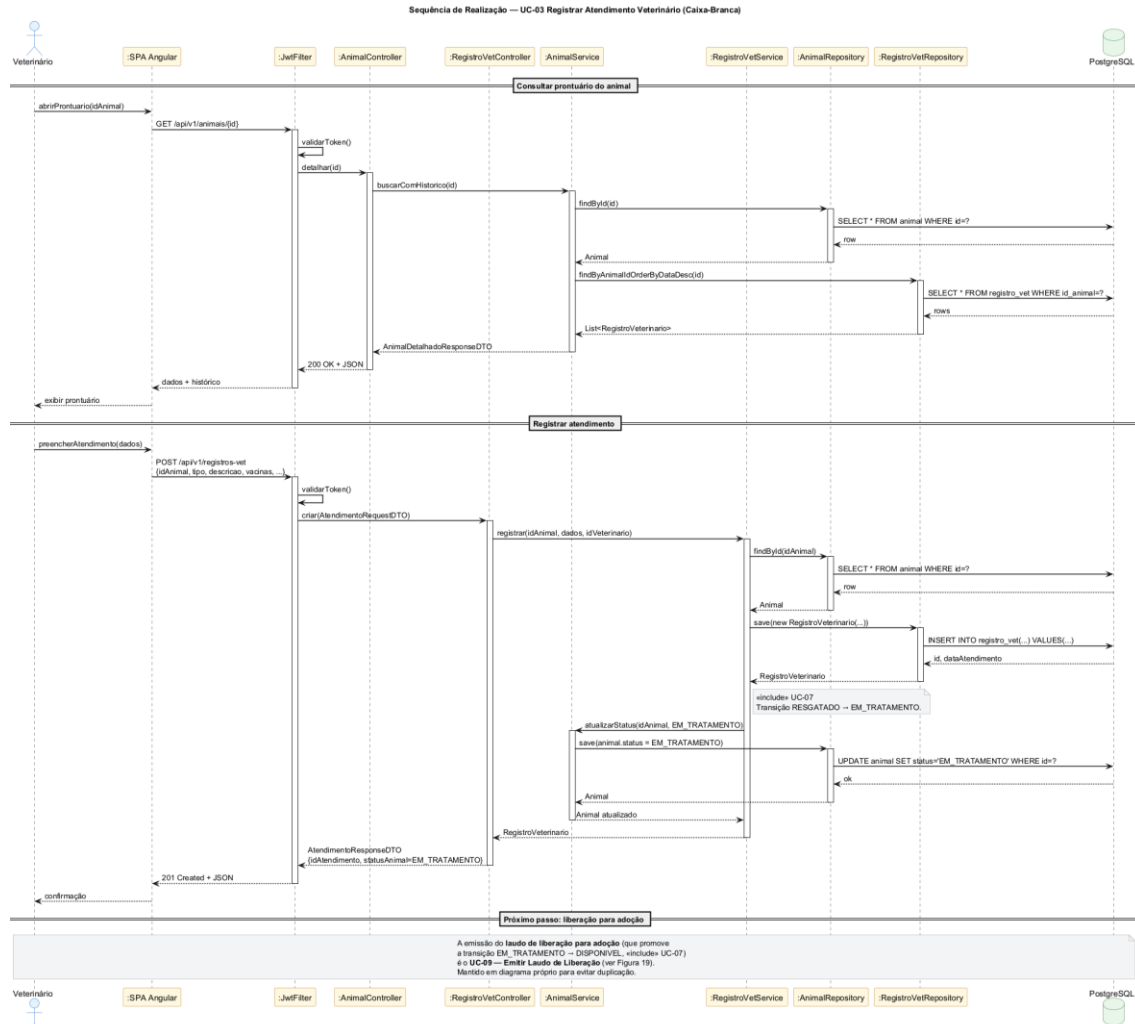


Figura 10 - Realizacao UC-03 Registrar Atendimento Veterinario

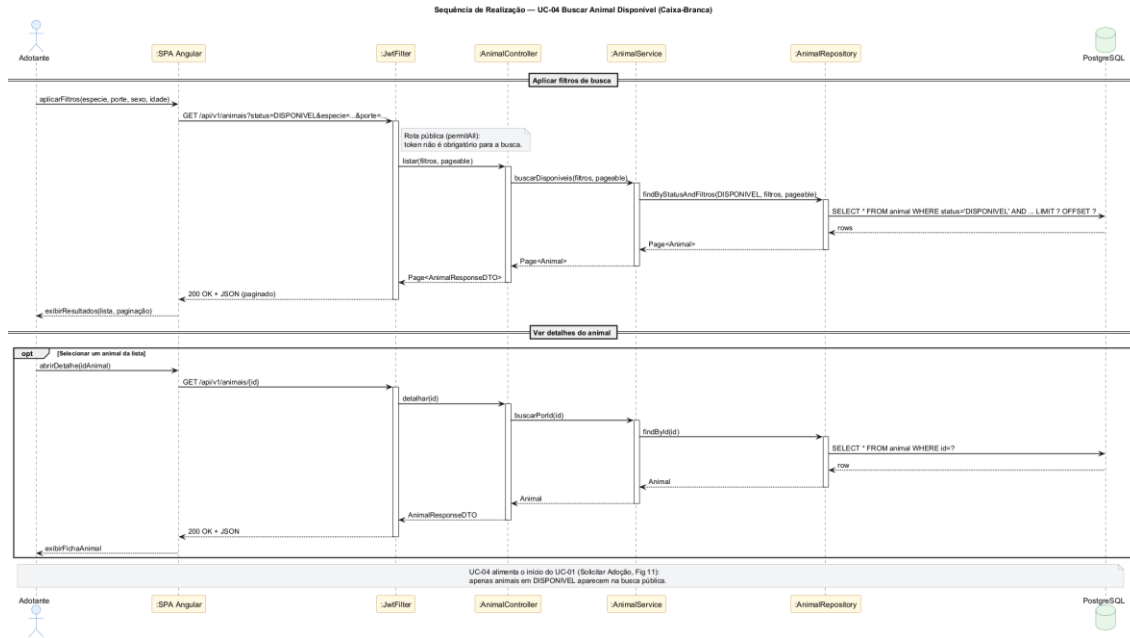


Figura 11 - Realizacao UC-04 Buscar Animal Disponível

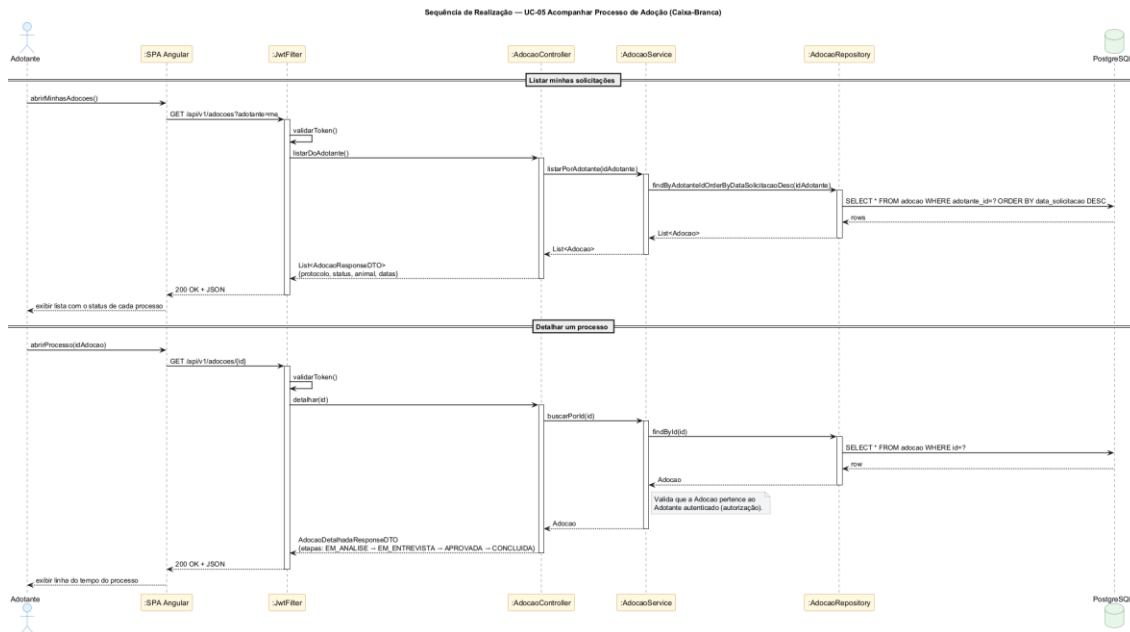


Figura 12 - Realizacao UC-05 Acompanhar Processo de Adocao

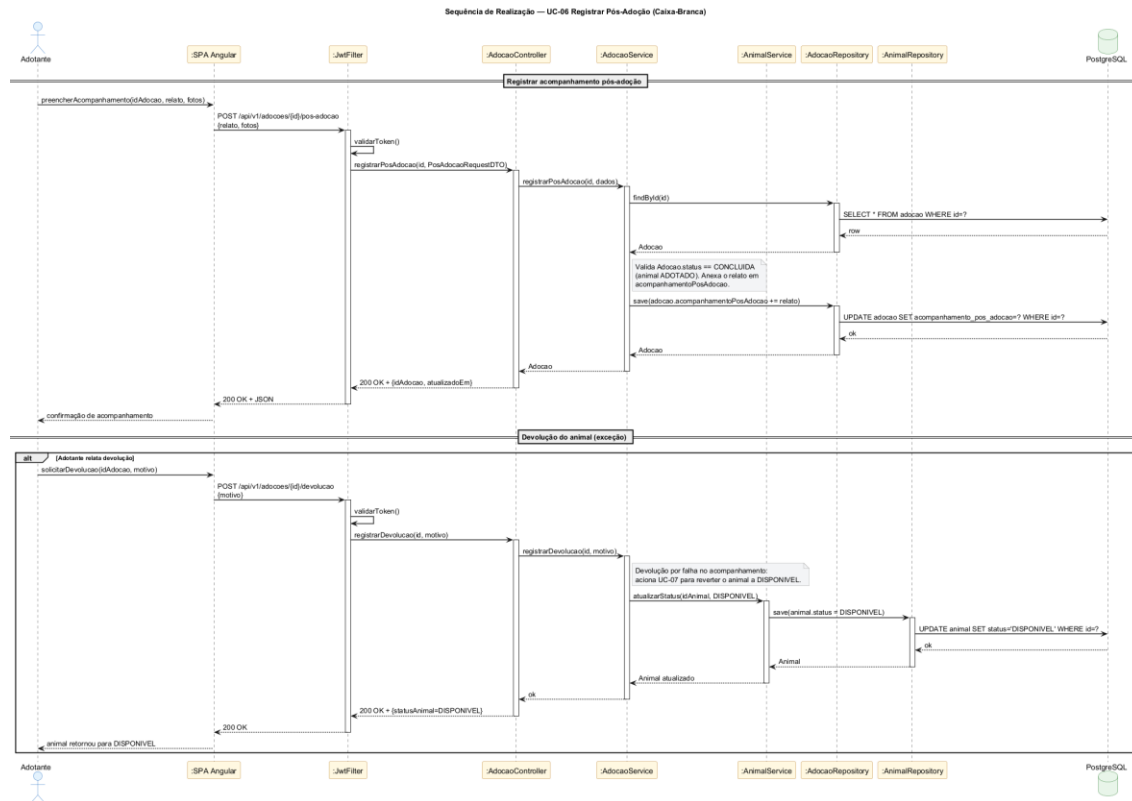


Figura 13 - Realizacao UC-06 Registrar Pos-Adocao

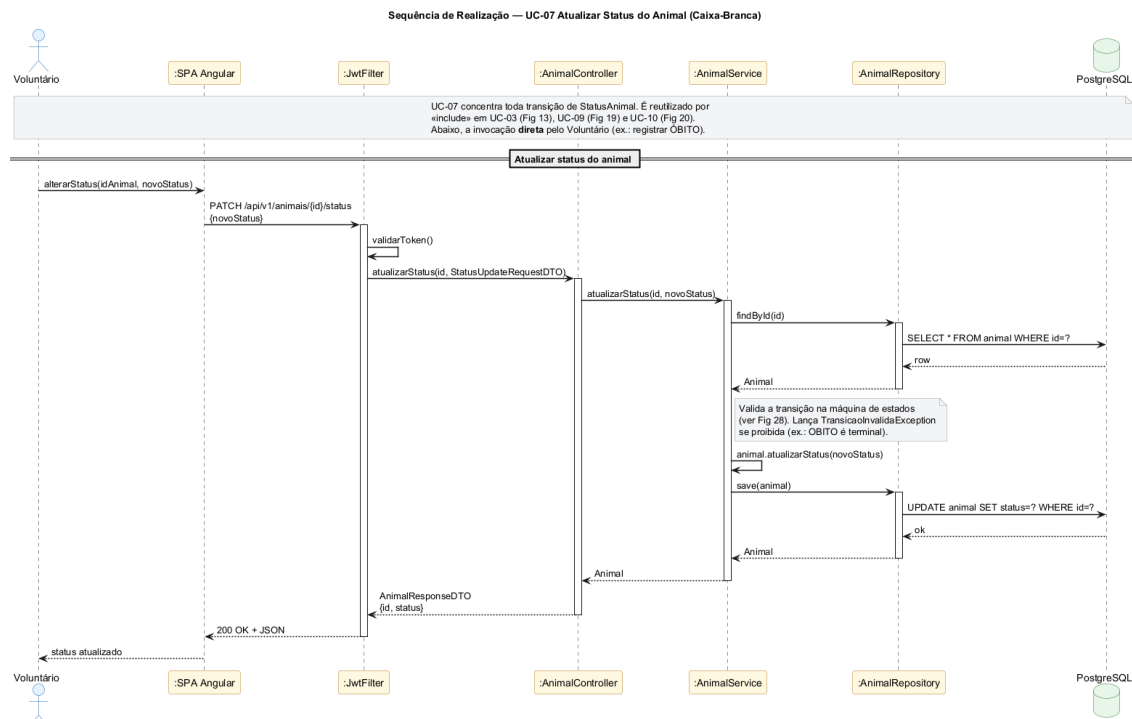


Figura 14 - Realizacao UC-07 Atualizar Status do Animal

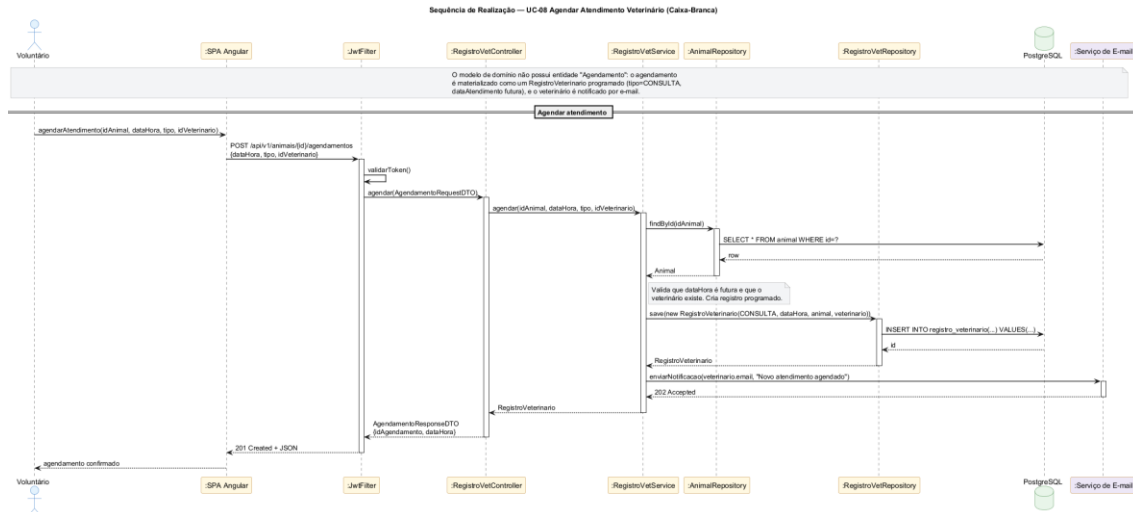


Figura 15 - Realizacao UC-08 Agendar Atendimento Veterinario

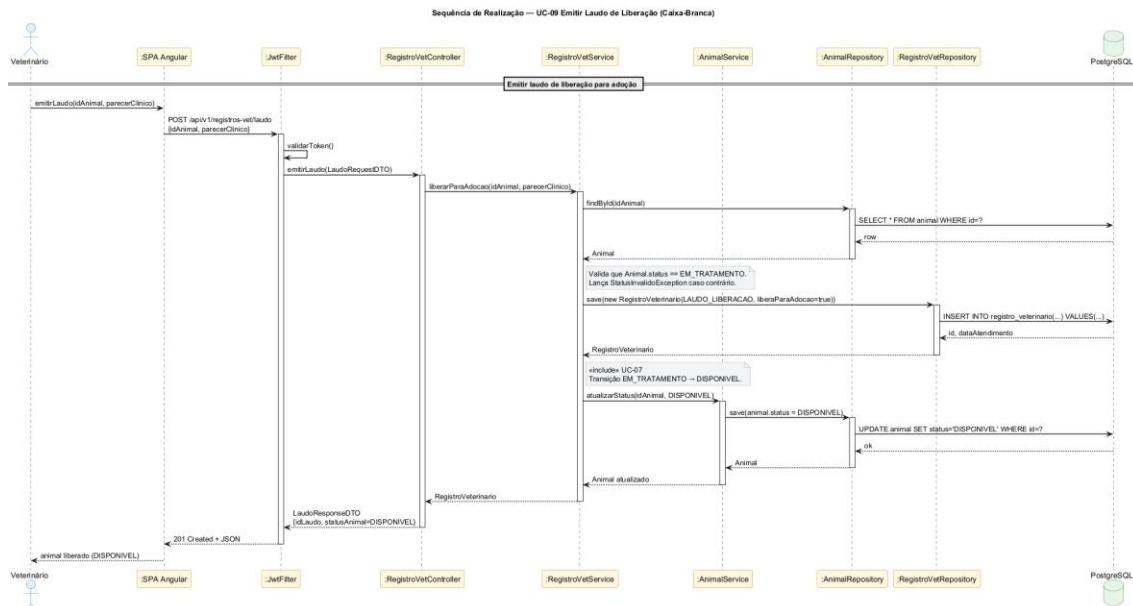


Figura 16 - Realizacao UC-09 Emitir Laudo de Liberacao para Adocao



Figura 17 - Realizacao UC-10 Aprovar/Rejeitar Adocao

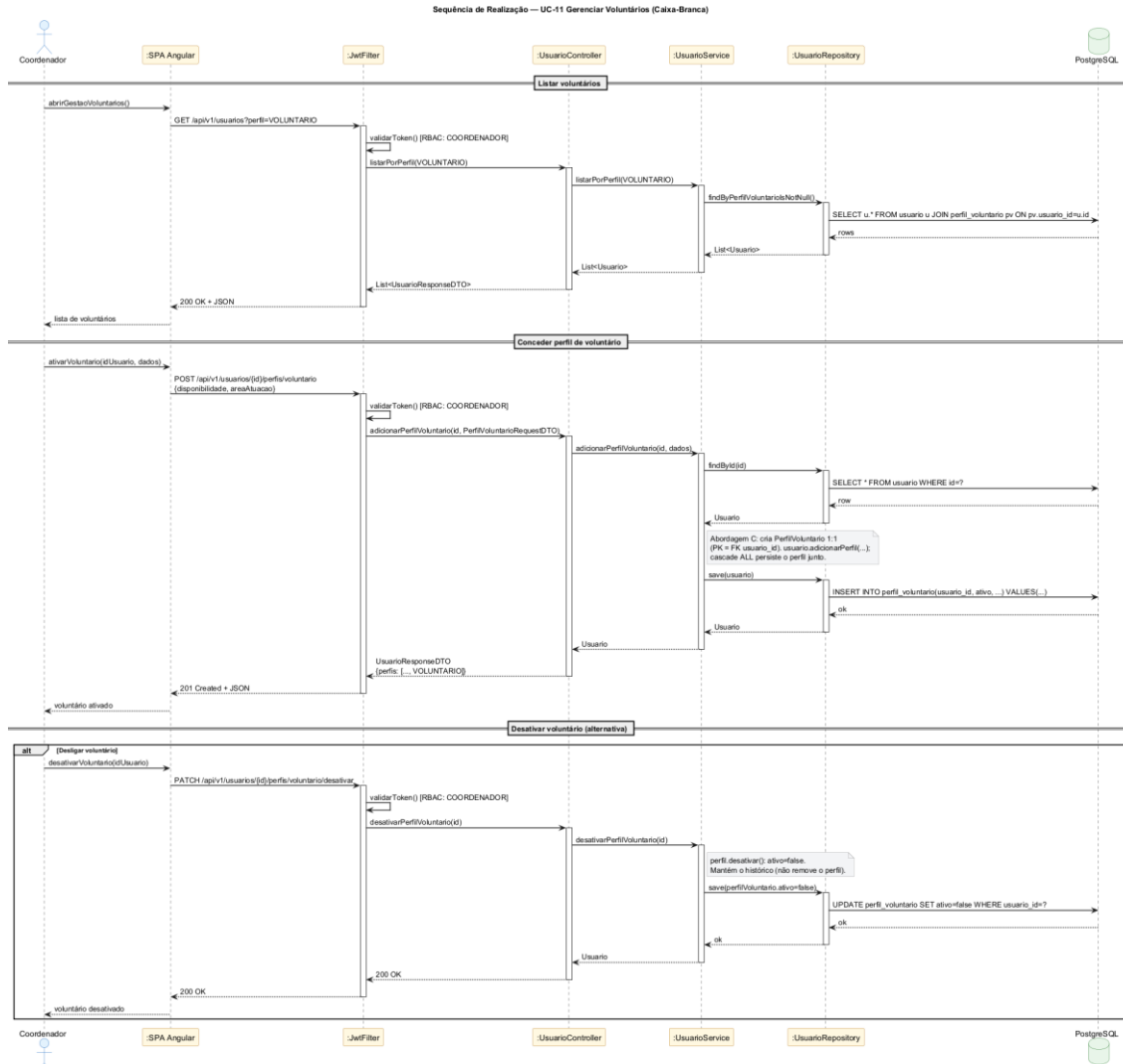


Figura 18 - Realização UC-11 Gerenciar Voluntários

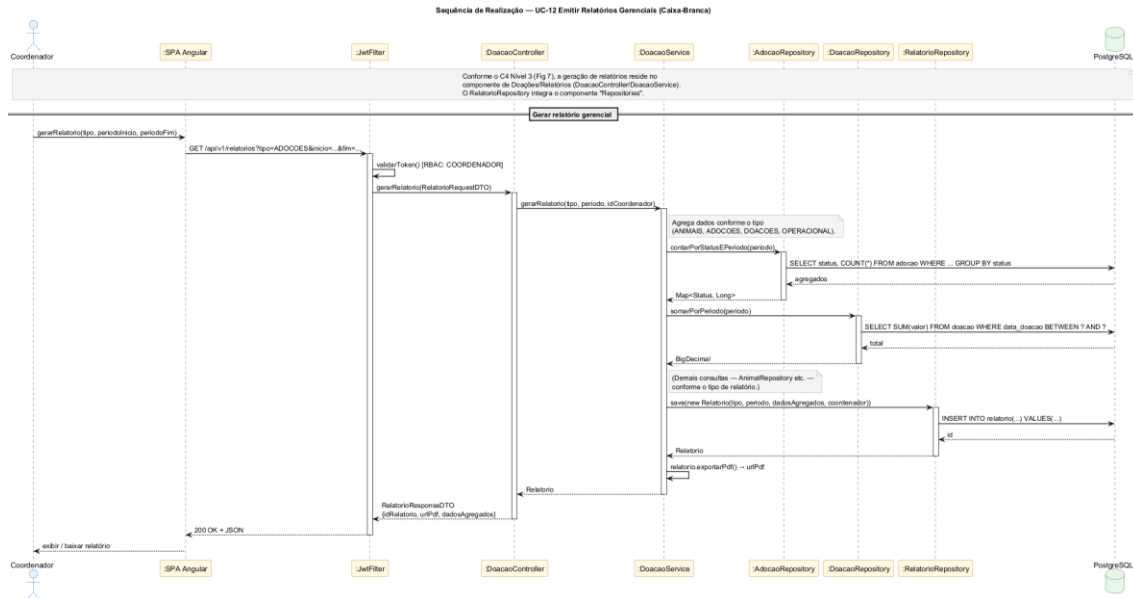


Figura 19 - Realizacao UC-12 Emitir Relatorios Gerenciais

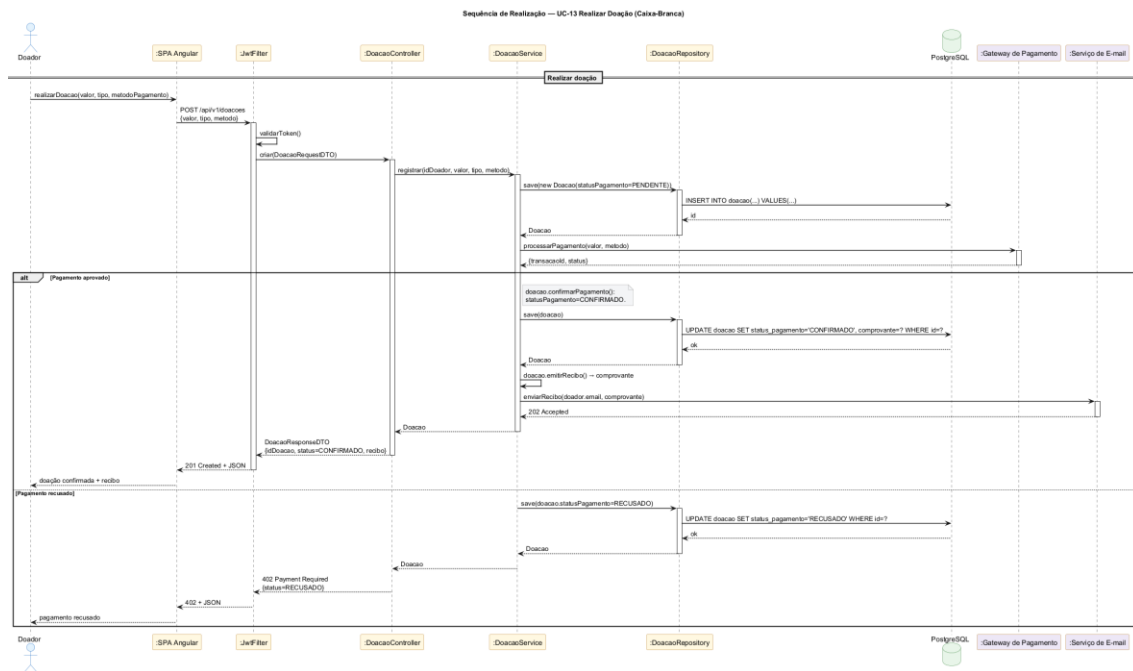


Figura 20 - Realizacao UC-13 Realizar Doacao

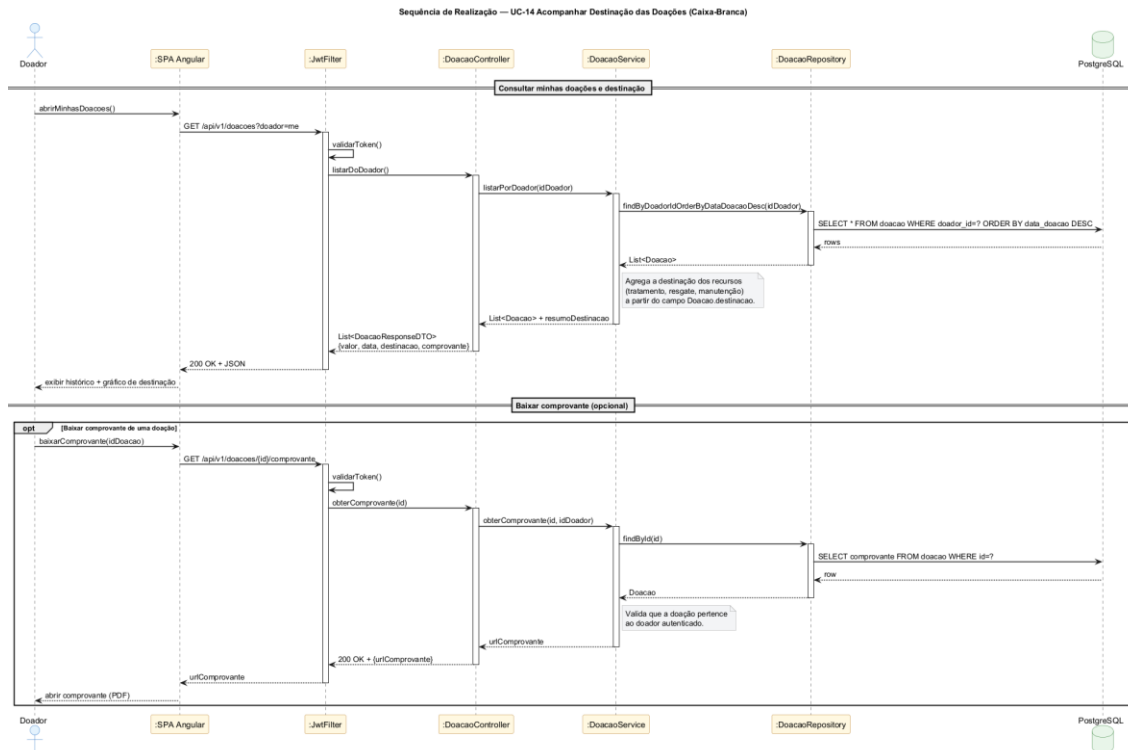


Figura 21 - Realizacao UC-14 Acompanhar Destinacao das Doacoes

Observacoes de modelagem:

- A autenticação é representada pelo passo `validarToken()` em `JwtFilter`, exigido em toda requisição autenticada.
- O UC-03 ilustra explicitamente o relacionamento «include» UC-07 (Atualizar Status do Animal) - `RegistroVetService` delega a `AnimalService.atualizarStatus(...)` após registrar o atendimento; o mesmo ocorre em UC-09 e UC-10.
- Validações de regra de negócio (ex.: `Animal.status == DISPONIVEL` antes de criar `Adocao`) aparecem como auto-mensagens ou notas no respectivo `Service`.
- O Diagrama de Sequência do Sistema (Figura 2) apresenta a visão caixa-preta ponta-a-ponta; cada operação de sistema é refinada (caixa-branca) no diagrama de sequência do respectivo caso de uso (Figuras 8 a 21).

3.5. Diagramas de Comunicação

Os diagramas de comunicação apresentam a mesma colaboração dos diagramas de sequência da Seção 3.4, porém vistos como rede de objetos com numeração explícita de mensagens em vez de eixo temporal vertical.

Aspecto	Sequência (Seção 3.4)	Comunicação (Seção 3.5)
Foco	Ordem temporal das mensagens	Vínculos estruturais entre objetos
Layout	Linhas de vida verticais	Objetos em rede
Ordenação	Posição vertical	Numeração explícita (1:, 2:, ...)

Cada par de objetos é conectado por uma aresta sem direção que carrega todas as mensagens trocadas entre eles - o sentido de cada mensagem é indicado pelos símbolos de direção conforme o padrão UML. Os repositórios *AnimalRepository* e *RegistroVetRepository* aparecem lado a lado para refletir que ambos colaboram diretamente com o PostgreSQL. Para o UC-03, o diagrama modela apenas a operação principal (*registrarAtendimento*) incluindo a transição «include» UC-07 (*VetSvc -> AnimSvc -> AnimRepo -> DB*) - o fluxo alternativo de emissão de laudo é coberto no diagrama de sequência (Figura 16).

Diagrama de Comunicacao - UC-01 Solicitar Adocao

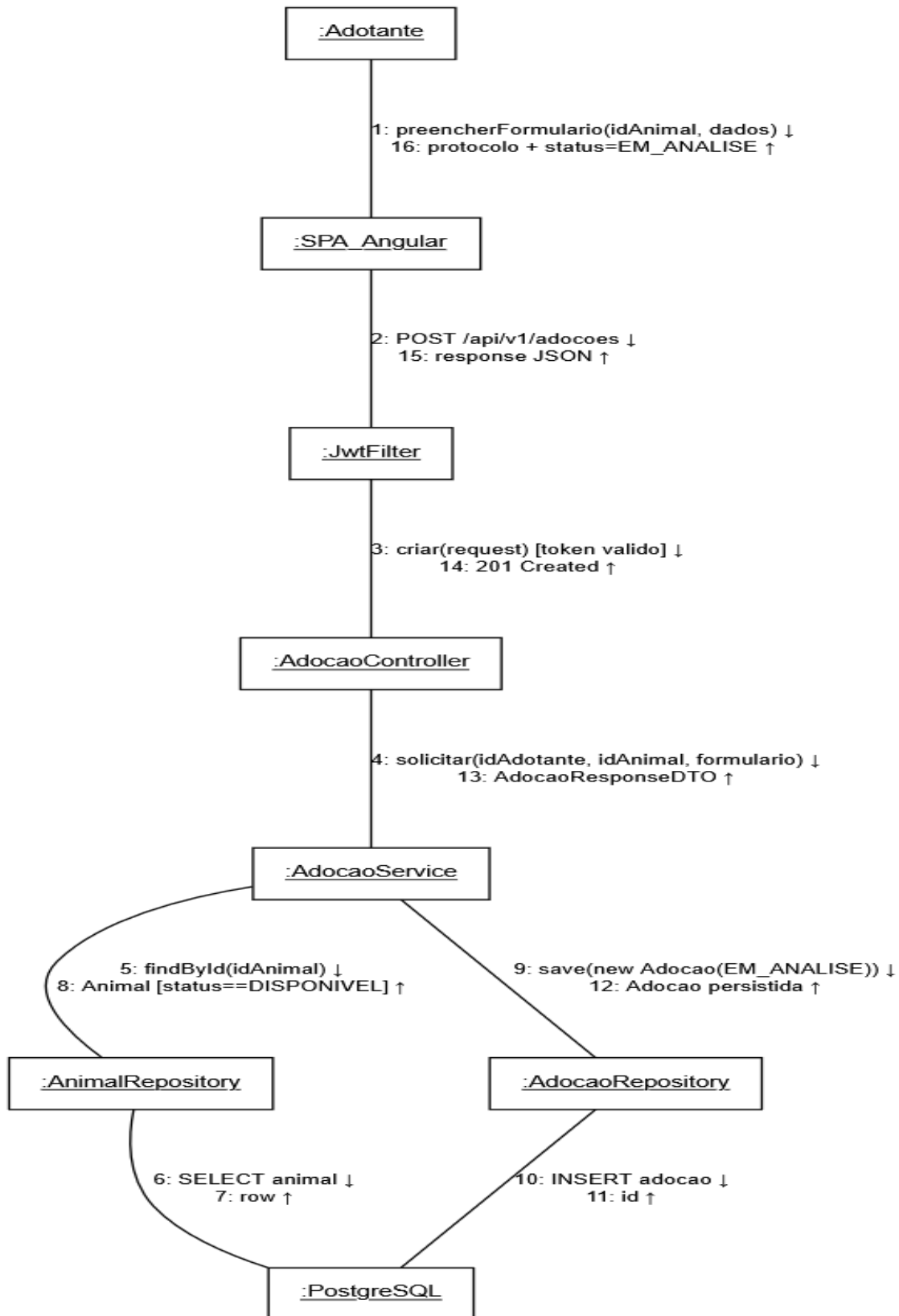


Figura 22 - Comunicacao UC-01 Solicitar Adocao

Diagrama de Comunicacao - UC-02 Registrar Animal Resgatado

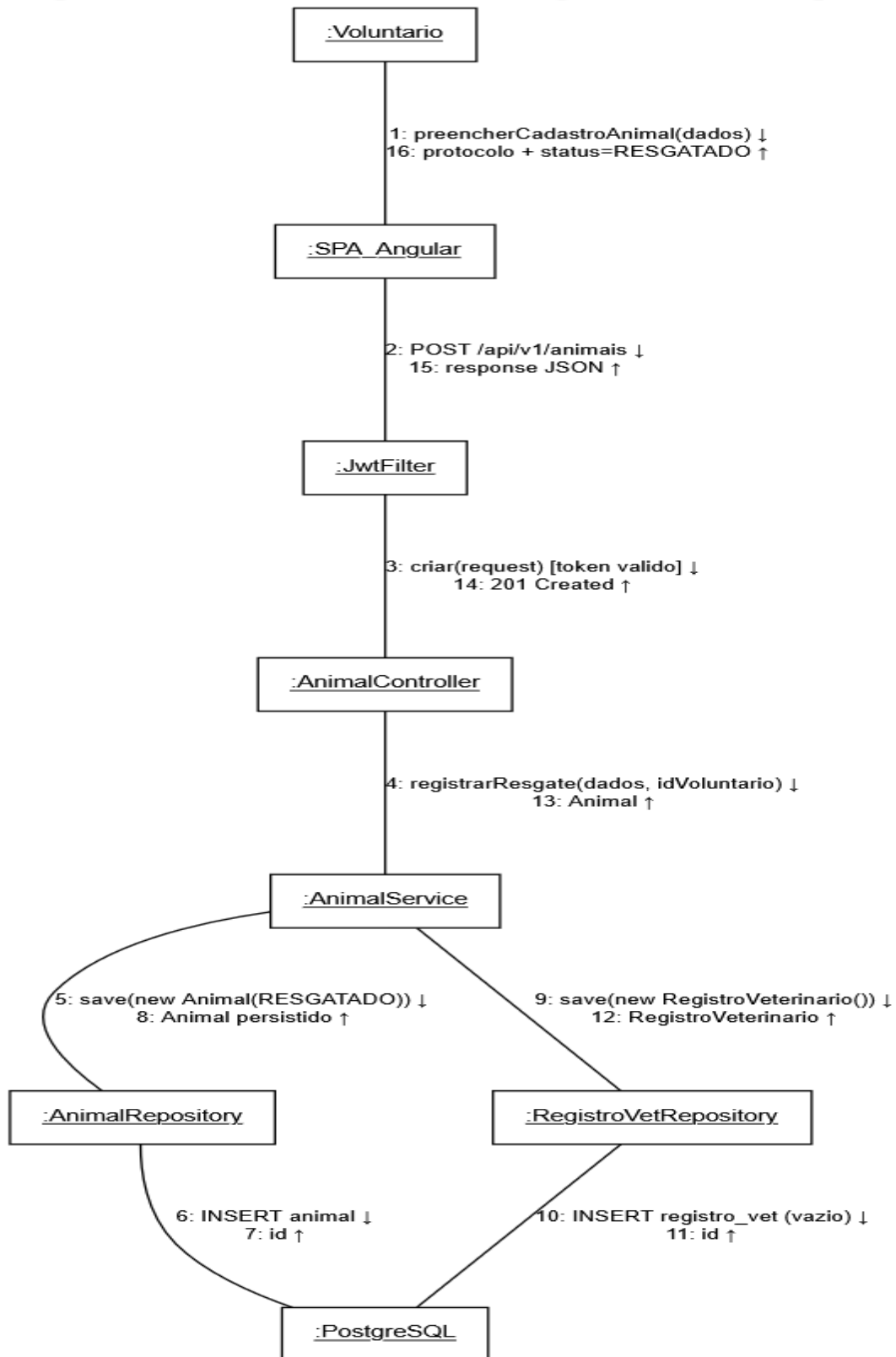


Figura 23 - Comunicacao UC-02 Registrar Animal Resgatado

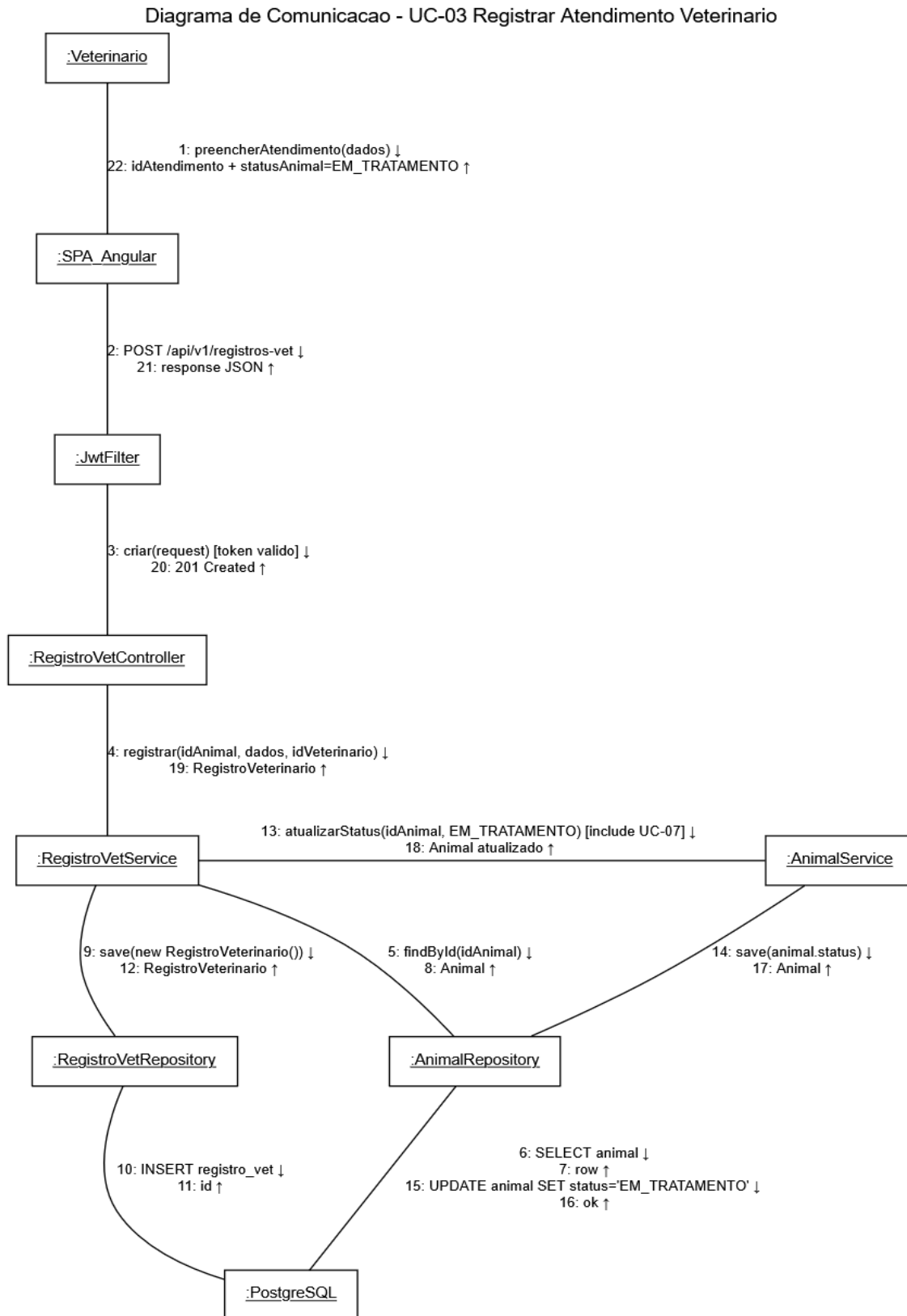


Figura 24 - Comunicacao UC-03 Registrar Atendimento Veterinario

3.6. Diagramas de Estados

Ciclo de vida de um animal dentro da plataforma, do resgate a adoção. Cada transição é acionada por uma operação de sistema ligada a um caso de uso da Seção 2.2 - em particular, todo o eixo principal de mudança de status é coberto por UC-07 Atualizar Status do Animal, que aparece como «include» nos UCs 03, 09 e 10.

3.6.1. Resumo dos estados

Estado	Significado	Proximo(s) estado(s) típico(s)
RESGATADO	Animal recém-resgatado, aguardando triagem veterinária	EM_TRATAMENTO, OBITO
EM_TRATAMENTO	Atendimentos veterinários em andamento (consultas, vacinas, cirurgias)	EM_TRATAMENTO (novo atendimento), DISPONIVEL, OBITO
DISPONIVEL	Apto a adoção, visível na busca pública (UC-04)	EM_PROCESSO_ADOCAO, OBITO
EM_PROCESSO_ADOCAO	Adoção aprovada, aguardando contrato e entrega	ADOTADO, DISPONIVEL, OBITO
ADOTADO	Adoção concluída, acompanhamento pós-adoção ativo	DISPONIVEL (devolução), estado terminal
OBITO	Animal veio a óbito - estado terminal	-

3.6.2. Principais transições

De -> Para	Evento / Operação	Caso de Uso
RESGATADO -> EM_TRATAMENTO	Primeiro registrarAtendimento() após resgate	UC-03 «include» UC-07
EM_TRATAMENTO -> EM_TRATAMENTO	Novo atendimento (auto-transição)	UC-03
EM_TRATAMENTO -> DISPONIVEL	emitirLaudoLiberacao() com liberaParaAdocao = true	UC-09 «include» UC-07
DISPONIVEL -> EM_PROCESSO_ADOCAO	aprovarAdocao() pelo Coordenador	UC-10 «include» UC-07
EM_PROCESSO_ADOCAO -> ADOTADO	concluirAdocao() (contrato assinado)	UC-10
EM_PROCESSO_ADOCAO -> DISPONIVEL	Desistência do adotante ou rejeição	UC-10

ADOTADO -> DISPONIVEL	Devolucao do animal (falha no acompanhamento)	UC-06
qualquer -> OBITO	Obito clinico do animal	UC-03

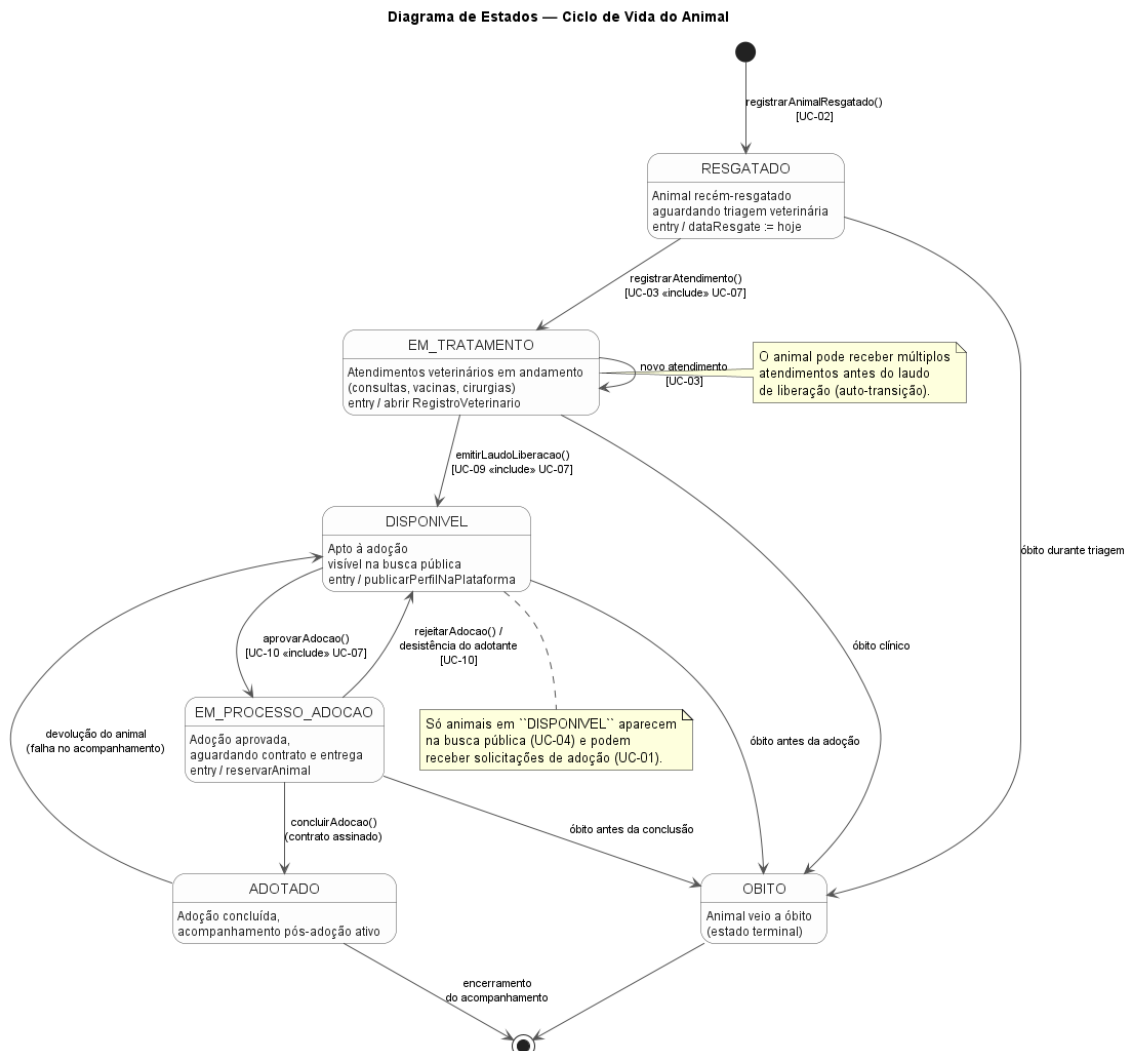


Figura 25 - Diagrama de Estados: Ciclo de Vida do Animal

4. Modelos de Dados

Apresentam-se os esquemas de banco de dados do PataAmiga e a estratégia de mapeamento entre as representações de objetos (entidades JPA) e não-objetos (tabelas relacionais do PostgreSQL).

O modelo lógico espelha o diagrama de classes da Seção 3.3, preservando a Abordagem C para usuários: tabela usuário central e cinco tabelas de perfil opcionais 1:1.

4.1. Diagrama Entidade-Relacionamento

4.1.1. Visão geral das tabelas

Tabela	Cardinalidade com usuario	Papel
usuario	-	Identidade central; dados pessoais e credenciais
perfil_adotante	1 - 0..1	Dados específicos do papel Adotante
perfil_doador	1 - 0..1	Dados específicos do papel Doador
perfil_voluntario	1 - 0..1	Dados específicos do papel Voluntario
perfil_veterinario	1 - 0..1	Dados específicos do papel Veterinario (CRMV, especialidade)
perfil_coordenador	1 - 0..1	Dados específicos do papel Coordenador
animal	-	Entidade central do negocio
registro_veterinario	-	Prontuario; FK para animal e perfil veterinario
adocao	-	Processo de adocao; FKs para animal, perfil_adotante e perfil_coordenador
doacao	-	Aporte financeiro; FK para perfil_doador
relatorio	-	Saida agregada; FK para perfil_coordenador

Nas cinco tabelas perfil_* a coluna usuario_id e simultaneamente PK e FK para usuario(id) - característica da Abordagem C que garante a cardinalidade 1:1 opcional sem coluna tipo_perfil discriminadora e sem heranca no banco.

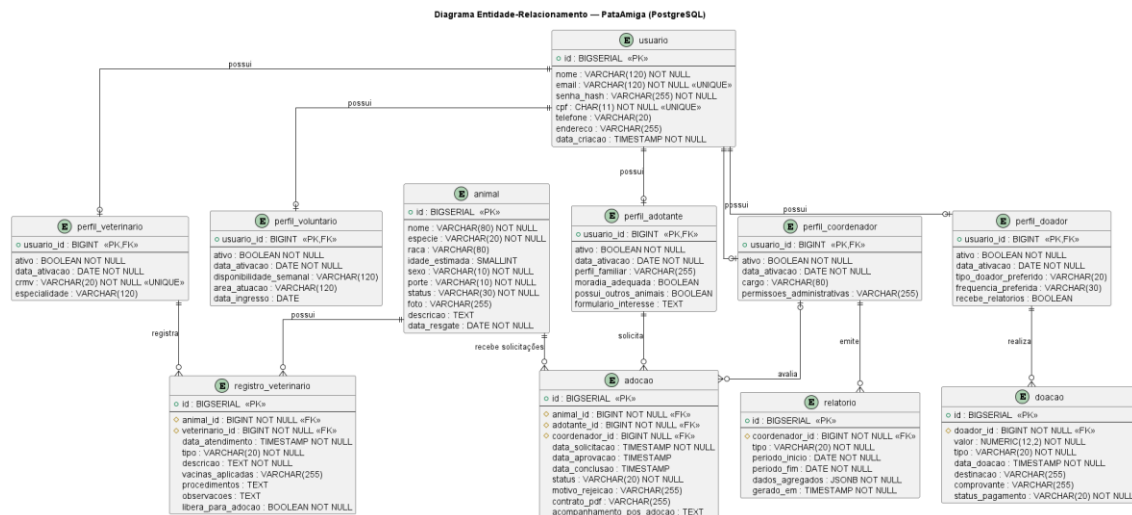


Figura 26 - Diagrama Entidade-Relacionamento do PataAmiga (PostgreSQL)

4.2. Estratégia de Mapeamento Objeto-Relacional

O backend usa Hibernate como provider JPA. Cada classe do modelo de domínio (Seção 3.3) é mapeada para uma tabela PostgreSQL conforme as convenções a seguir.

4.2.1. Convenções gerais

Aspecto	Decisão
Nomes de tabelas e colunas	snake_case minúsculo (via <code>Hibernate.physical_naming_strategy = CamelCaseToUnderscoresNamingStrategy</code>)
Chaves primárias	<code>@Id @GeneratedValue(strategy = GenerationType.IDENTITY)</code> mapeado para <code>BIGSERIAL</code>
Enums	<code>@Enumerated(EnumType.STRING)</code> - preserva legibilidade no banco e segurança em refatorações
Datas/horas	<code>LocalDate</code> -> <code>DATE</code> , <code>LocalDateTime</code> -> <code>TIMESTAMP</code> (JPA 3.1, sem <code>@Temporal</code>)
Valores monetários	<code>BigDecimal</code> -> <code>NUMERIC(12,2)</code> em <code>Doacao.valor</code>
Texto longo	Campos como <code>Adocao.acompanhamentoPosAdocao</code> , <code>Animal.descricao</code> e <code>RegistroVeterinarioobservacoes</code> usam <code>@Column(columnDefinition = "TEXT")</code>
JSON estruturado	<code>Relatorio.dadosAgregados</code> mapeado como <code>JSONB</code> via <code>@JdbcTypeCode(SqlTypes.JSON)</code> (Hibernate 6)
<code>equals</code> / <code>hashCode</code>	Baseados apenas no id (entidades JPA com identidade gerada)
Versionamento	<code>@Version</code> em entidades editáveis com concorrência (<code>Animal</code> , <code>Adocao</code>) para locking otimista

4.2.2. Mapeamento da Abordagem C (Usuario + Perfis)

Cada perfil é uma entidade independente cuja chave primária é também a chave estrangeira para usuário, materializando o relacionamento 1:1 opcional sem coluna nullable na tabela base.

```
@Entity
@Table(name = "usuario")
public class Usuario {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```

@Column(nullable = false, unique = true, length = 120)
private String email;

@OneToOne(mappedBy = "usuario", cascade = CascadeType.ALL,
            orphanRemoval = true, fetch = FetchType.LAZY)
private PerfilAdotante perfilAdotante;
// ... mesma anotacao para os outros 4 perfis
}

@Entity
@Table(name = "perfil_adotante")
public class PerfilAdotante {
    @Id
    private Long id;           // mesmo valor de usuario.id

    @MapsId
    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "usuario_id")
    private Usuario usuario;
}

```

- @MapsId informa ao Hibernate que a PK do PerfilAdotante é derivada da FK para Usuario - não há sequência separada.
- O lado proprietário do relacionamento é o perfil (ele carrega usuario_id); o Usuario usa mappedBy.
- cascade = ALL + orphanRemoval = true permitem criar/remover o perfil junto com o usuario quando apropriado.

Por que não @Inheritance? Uma hierarquia JPA (SINGLE_TABLE, JOINED ou TABLE_PER_CLASS) exigiria que cada usuario fosse exatamente um tipo de perfil. No PataAmiga uma mesma pessoa pode acumular qualquer combinação de papéis (ex.: voluntário que também adota e doa), o que invalida a herança e justifica a Abordagem C com composição.

4.2.3. Mapeamento dos principais relacionamentos

Relacionamento (UML)	Anotações JPA	Coluna FK no banco
Animal "1" -> "0..*" RegistroVeterinario	@OneToMany(mappedBy="animal", cascade=ALL, orphanRemoval=true) em Animal; @ManyToOne @JoinColumn(name="animal_id") em RegistroVeterinario	registro_veterinario.animal_id
PerfilVeterinario "1" -> "0..*" RegistroVeterinario	@ManyToOne(fetch=LAZY) @JoinColumn(name="veterinario_id") em RegistroVeterinario	registro_veterinario.veterinario_id
Animal "1" -> "0..*" Adocao	@OneToMany(mappedBy="animal") em Animal; @ManyToOne @JoinColumn(name="animal_id") em Adocao	adocao.animal_id

PerfilAdotante "1" -> "0..*" Adocao	@ManyToOne @JoinColumn(name="adotante_id") em Adocao	adocao.adotante_id
PerfilCoordenador "0..1" -> "0..*" Adocao	@ManyToOne(optional=true) @JoinColumn(name="coordenador_id") em Adocao	adocao.coordenador_id (nullable)
PerfilDoador "1" -> "0..*" Doacao	@ManyToOne @JoinColumn(name="doador_id") em Doacao	doacao.doador_id
PerfilCoordenador "1" -> "0..*" Relatorio	@ManyToOne @JoinColumn(name="coordenador_id") em Relatorio	relatorio.coordenador_id

4.2.4. Estratégias de busca (FetchType) e cascata

- Default = LAZY em todos os @ManyToOne e @OneToOne - colecoes e referencias so sao carregadas sob demanda, evitando o N+1. Quando uma view precisa de joins, a camada Service usa @EntityGraph ou JPQL com JOIN FETCH.
- CascadeType.ALL + orphanRemoval = true apenas onde a composicao e exclusiva: Usuario -> Perfil*, Animal -> RegistroVeterinario. Nas demais relacoes usa-se CascadeType.PERSIST ou nenhuma cascata, evitando exclusoes em cadeia indesejadas.

4.2.5. Versionamento de schema

Migrations versionadas com Flyway (db/migration/V<ts>__<descricao>.sql). O Hibernate roda em modo ddl-auto=validate em producao - o schema e construido exclusivamente pelas migrations, e a aplicacao apenas valida na inicializacao. Isso garante que o ER documentado nesta secao corresponde ao banco real.